Orbital Dragons to Hunt Cryptids: Utilizing Non-Euclidean Geometry to Solve Busy Beaver

Enan Srivastava

DeepMath 2026 — Extended Abstract

Abstract

The busy beaver value $BB(n)$ is the longest finite runtime of any n-state Turing machine started on a blank tape. $BB(5)$ was settled in 2024 with a computer-checked proof. $BB(6)$ is blocked by a handful of machines whose tapes secretly run Collatz-style arithmetic; nobody can prove whether they stop. We study two of these machines. Our approach is empirical first: we log the machine's behavior as a sequence of events, embed the event data in logarithmic, simplex, and hyperbolic coordinates, and find that the dynamics are piecewise affine in a five-integer state. We then prove this description correct in the Lean proof assistant, down to the machine's individual steps and with a minimal axiom footprint. The result is a machine-checked theorem: if a simple five-integer recurrence never enters an explicit finite set of bad configurations, the machine never halts. We also prove the converse pressure: entering a bad configuration forces the machine to halt. The open problem is thereby moved, without loss, from a Turing machine to a short arithmetic recurrence. Every empirical claim is validated against 10^9 raw machine steps, and the recurrence is tested for 10^8 iterations.

1 The problem, from zero

A Turing machine here is a finite table: six control states, a tape of 0s and 1s, and for each (state, read symbol) pair an instruction — write a bit, move left or right, change state — or nothing. If the table has no instruction for the current situation, the machine halts. $BB(n)$ asks: among all n-state tables that eventually halt from the all-zero tape, what is the longest runtime? The function is noncomputable, so each new value is won machine by machine: every table either halts (run it) or must be *proven* to run forever.

The bbchallenge collaboration settled $BB(5) = 47{,}176{,}870$ with a proof checked in the Rocq proof assistant [1, 2]. At six states a residual set of machines defeats all known automated deciders. The community calls them cryptids. The reason they are hard is concrete: their tape dynamics implement iterated arithmetic maps of the same character as the Collatz $(3x+1)$ map — deterministic integer maps where parity decides the next move, and where the question is whether one specific orbit ever hits a small bad set. The strongest known result for such maps is Tao's: *almost all* Collatz orbits reach almost bounded values [3]. That theorem is about typical orbits. Busy beaver questions need the worst case: one orbit, followed forever. No general tool exists for that, which is why each cryptid is studied on its own terms.

We work on two six-state cryptids, a mirror-image pair we call dossier-2 and dossier-3. Machine one is `1RB1RC_1LC1LC_1RE0LD_1LB1RF_---1RA_0RD1RD` in the standard table notation.

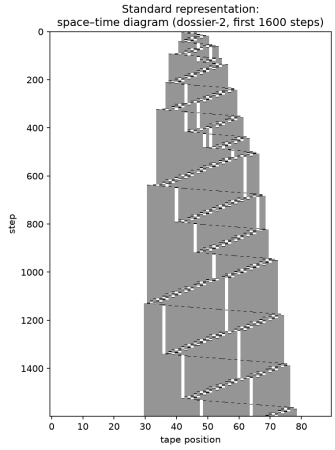


Figure 1: Raw view: the machine's first 1,600 steps (dark = 1s, one row per step). Nothing about the long-run law is visible here.

2 What we do

Our method has three stages. Watch, compress, certify.

Watch. We run the machine and record only the moments that matter. For these machines there is a natural clock: a recurring configuration we call a spawn entry. Between two spawn entries the machine performs one bounded "episode" of work. Sampling at spawn entries turns a raw trajectory of 10^9 steps into a clean sequence of a few million events.

Compress. At every spawn entry the tape has the same form: a solid block of 1s with exactly two 0-cells ("holes") inside it. So the whole configuration is five integers: the block's two endpoints l, r, the hole positions h_1, h_2, and a tag. We write $a = h_1 - l$, $b = h_2 - h_1$, $c = r - h_2$ for the three gap lengths. Plotted in raw coordinates, the event data looks like growth curves with no visible law (Figure 1). Plotted in the right coordinates — $\log(1+x)$ per component, or the projective simplex $(a, b, c)/(a+b+c)$, or shape against log-scale — the same data is piecewise linear (Figure 2). Straight segments are one regime; corners are regime changes. These coordinate choices are standard elsewhere: the log map is the tropical limit of algebraic geometry [6, 7], the simplex normalization is Perron–Frobenius, and shape-versus-scale is the hyperbolic geometry used for hierarchical data [8]. The underlying view of a Turing machine as a dynamical system goes back to Moore and Kůrka [4, 5].

Certify. The picture suggests a law; the law must then be proven about the actual machine, or it is only a picture. We first validate it empirically and fail-closed: a predictor that knows the conjectured law must reproduce every event — exact positions, exact tape — over 10^9 raw steps per machine, and any miss aborts. It reproduces all of them. We then prove the law in Lean 4, using only the kernel's minimal axioms (propext, Quot.sound; no classical choice). The certified account consists of: seven theorems, one per episode type, each giving the episode's exact step count, exact output shape, and safety; two inductions that handle the episodes' unbounded middle parts; a concrete certificate for the machine's startup (235 steps for one machine, 2,044 for the other); and a transport theorem showing that a five-integer function metaStep, iterated from the startup state, predicts the machine forever.

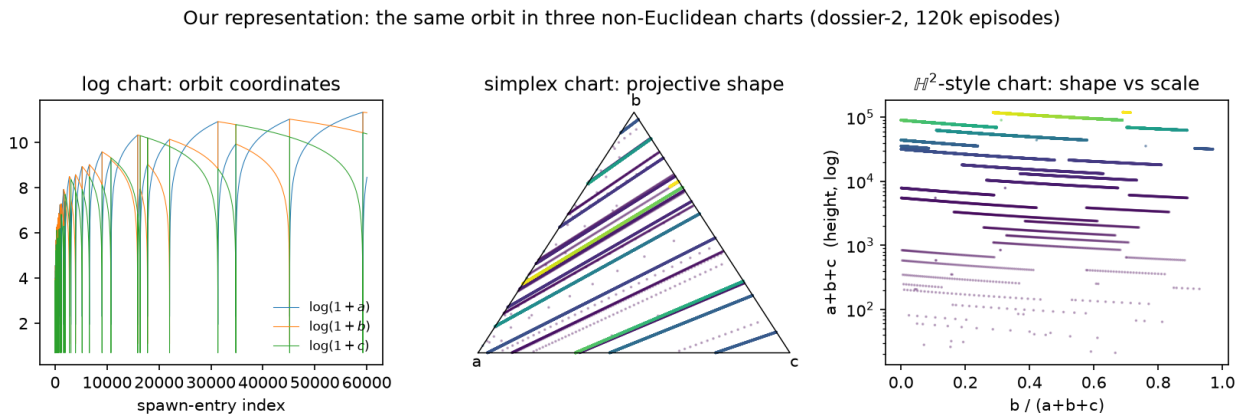


Figure 2: The same machine, 60,049 events, in three coordinate systems. Left: componentwise $\log(1+x)$; each straight segment is one regime of the dynamics. Center: the simplex $(a, b, c)/(a+b+c)$; scale is removed and only shape moves. Right: shape against log-scale; the orbit climbs one zig per regime change.

3 The result

Halting, for these machines, can only happen one way: a specific control state must read a 0 with another 0 beside it. The certified episode theorems show this situation never arises while the five-integer iteration stays inside an explicit guarded region. Writing SafeMetaOrbit for the statement that the iteration never leaves that region:

Theorem 1. *For each machine, checked by the Lean kernel at axioms* {propext, Quot.sound}*:* SafeMetaOrbit $\to \neg$RawHalts(blankStart).

Two facts make this reduction meaningful rather than cosmetic. First, it is complete: startup, one exceptional episode, and every future step of the machine are covered by theorems, so nothing about the machine remains to be understood — only the arithmetic. Second, it is sharp: we constructed the configurations just outside the guarded region and ran the machine from them; it halts within about fifty steps. Leaving the region is not merely unproven-safe. It is fatal. So SafeMetaOrbit *is* the halting problem, restated as arithmetic.

What kind of arithmetic? Inside the main regime the iteration is $(a, b, c) \mapsto (a{+}6,\ b{-}2,\ c{-}2)$, so $\gamma = c - b$ is conserved while the counters drain. When b or c runs out, a renewal move re-seeds the triple by subtracting and swapping counters, with parity choosing the branch — part Euclidean algorithm, part Collatz. Figure 3 shows the value of γ at the first fifty renewals against the fatal band $\gamma \in \{1, 3, 5\}$. The recurrence avoids the band for 10^8 iterations (about 10^{15} machine steps). Proving it does so forever is the open problem, now in a form a number theorist can hold in one hand.

4 Why this may interest this community

Two reasons. The first is the workflow. This project is an instance of the emerging human–AI–kernel pattern in mathematics — AI systems draft and compile proofs at scale, a proof kernel arbitrates, humans own claims — the pattern of Tao's Equational Theories Project and the new Palomar registry for Lean-verified results [9, 10]. All of our Lean was drafted by an AI assistant from

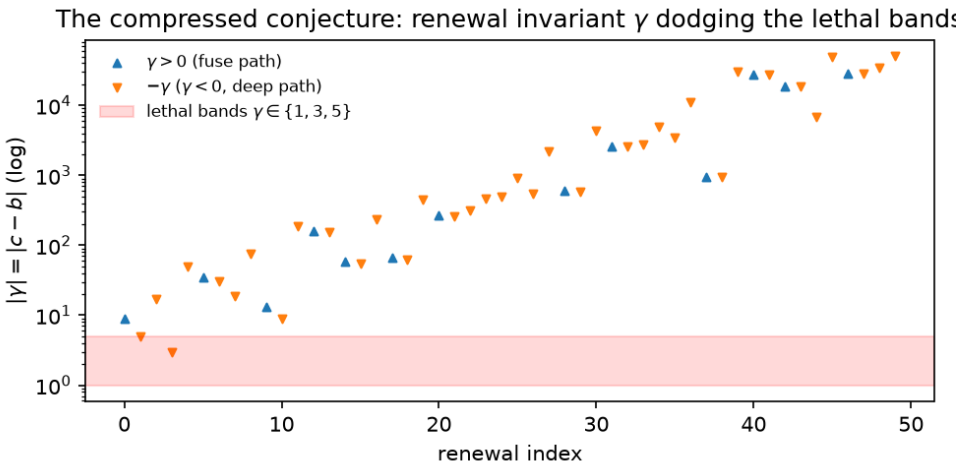


Figure 3: The reduced problem. Each point is one renewal of the recurrence; the shaded band is proven fatal. The conjecture: the process never enters the band.

validated empirical specifications; the kernel checked roughly fifteen thousand lines; the generators are convenience, not trust surface. The second reason is methodological kinship. Finding that a high-dimensional trajectory is governed by a low-dimensional piecewise-affine law, by choosing coordinates in which the structure is linear, is the daily business of representation learning. Here the same move is carried one step further: the discovered representation is not evaluated on held-out data but proven exactly correct, step by step, against the system that generated it.

Scope. We claim no unconditional non-halting result. The coordinate charts guide discovery and exposition; no theorem depends on them. Empirical horizons are what we ran, not what is true.

Acknowledgments. The Lean developments, empirical validation, and proof generators were built in collaboration with an AI assistant (Claude, Anthropic). Every proof is checked by the Lean 4 kernel at the stated axioms.

References

[1] The bbchallenge Collaboration. Determination of the fifth Busy Beaver value. arXiv:2509.12337.

[2] The bbchallenge Collaboration. Turing machine deciders, part I. arXiv:2504.20563.

[3] T. Tao. Almost all orbits of the Collatz map attain almost bounded values. arXiv:1909.03562.

[4] C. Moore. Generalized shifts: unpredictability and undecidability in dynamical systems. *Nonlinearity* 4 (1991).

[5] P. Kůrka. On topological dynamics of Turing machines. *Theoret. Comput. Sci.* 174 (1997).

[6] O. Viro. On basic concepts of tropical geometry. *Proc. Steklov Inst. Math.* 273 (2011).

[7] G. Litvinov. Maslov dequantization, idempotent and tropical mathematics. *J. Math. Sci.* 140 (2007).

[8] M. Nickel and D. Kiela. Poincaré embeddings for learning hierarchical representations. NeurIPS 2017.

[9] T. Tao et al. The Equational Theories Project. arXiv:2512.07087.

[10] T. Tao. Palomar: a registry of Lean-verified mathematics, 2026. `palomar-registry.org`.